

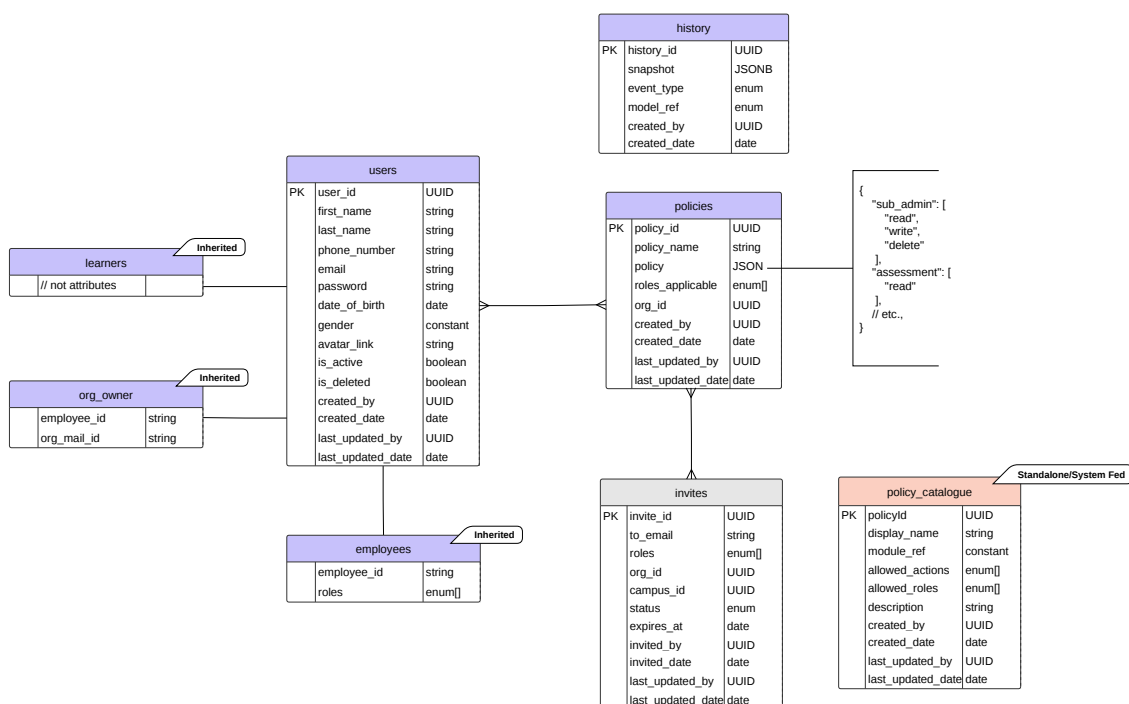
User-Service

This documentation provides a detailed reference for the entity-relationship diagram (ERD) presented, focusing on user management, organizational roles, policies, permissions, and onboarding/invitation workflows. Each table is described with its attributes, data types, relationships, and purposes. Many-to-many relationships and join tables are explicitly covered.



Issues:

- Inheritance is not well supported in PostgreSQL; Do not use it.
- Using Enums is not a great Idea, need to know why?



1. users

The central entity for all individuals in the system, storing login, profile, and status information. Forms the base for role inheritance (employee, learner, etc.).

Attribute	Data Type	Relation	Description
user_id	UUID	PK	Unique identifier for user
first_name	string		User's first name
last_name	string		User's last name
phone_number	string		Contact phone
email	string		Primary login/email
password	string		Hashed password
date_of_birth	date		Date of birth
gender	enum		Gender (enum type)
avatar_link	string		URL to avatar/profile image
is_active	boolean		Account active status
is_deleted	boolean		Indicates if account is soft-deleted
created_by	UUID	FK to users.user_id	Creator (admin or automated system)
created_date	date		User creation timestamp
last_updated_by	UUID	FK to users.user_id	Last updater
last_updated_date	date		Last update timestamp

2. learners

Specialized user type representing students/learners. No attributes currently, but will help in future extensibility.

Attribute	Data Type	Relation	Description
--	--	--	– no attributes currently

3. employees

Specialized user type representing platform staff, admins, mentors, etc.

Attribute	Data Type	Relation	Description
employee_id	string	PK	Unique in organization; may mirror user_id

Attribute	Data Type	Relation	Description
roles	enum[]		Array of roles held (mentor, admin, etc.)

4. org_owner

Specialized employee with organizational ownership.

Attribute	Data Type	Relation	Description
employee_id	string	FK to employees	Corresponds to employees table
org_mail_id	string		Official org-issued email

5. policies

Represents a group/set of permissions (policy group) applicable to users or roles. The `policy` field uses JSON for storing permissions mapping.

Attribute	Data Type	Relation	Description
policy_id	UUID	PK	Unique policy identifier
policy_name	string		Human-readable policy set name
policy	JSON		Detailed permissions (roles/actions map)
roles_applicable	enum[]		Roles that this policy can be assigned to
org_id	UUID		Organization context (if applicable)
created_by	UUID	FK to users.user_id	Policy creator
created_date	date		Creation timestamp
last_updated_by	UUID	FK to users.user_id	Who last updated
last_updated_date	date		Last updated at

6. invites

Tracks user onboarding invitations, the assigned role(s), and assigned policies at invite time.

Attribute	Data Type	Relation	Description
invite_id	UUID	PK	Unique invite identifier
to_email	string		Email address for invitation
roles	enum[]		Array of roles invited to
org_id	UUID		Organization where user will join
campus_id	UUID		(If applicable) campus location
status	enum		Invitation workflow state
expires_at	date		Expiry date
invited_by	UUID	FK to users.user_id	Who sent invitation
policy_id	UUID	FK to policies.policy_id (owning side)	Policy assigned at invite
created_date	date		Date invite created
last_updated_by	UUID	FK to users.user_id	Who last updated
last_updated_date	date		Last updated at

7. policy_catalogue

Master reference for all fine-grained (atomic) policies, modules, and allowable actions that policies can be built from.

Attribute	Data Type	Relation	Description
policyId	UUID	PK	Identifier for the atomic policy
display_name	string		Human-friendly policy/module name
module_ref	constant		System/module identifier
allowed_actions	enum[]		What actions are possible (read, write, etc.)
allowed_roles	enum[]		Roles this policy is available for
description	string		What this policy/module enables

Attribute	Data Type	Relation	Description
created_by	UUID	FK to users.user_id	Creator
created_date	date		Creation timestamp
last_updated_by	UUID	FK to users.user_id	Who last updated
last_updated_date	date		Last updated at

8. history

Tracks changes or significant events for auditable entities (users, policies, etc.).

Attribute	Data Type	Relation	Description
history_id	UUID	PK	Unique change event ID
snapshot	JSONB		Old values or complete record at change time
event_type	enum		Insert, update, delete, etc.
model_ref	enum		Which model/table this record is for
created_by	UUID	FK to users.user_id	Who made the change
created_date	date		When change occurred

Many-to-Many Relationships and Join Tables

a) policies ↔ users (M:N)

Purpose: Users can have multiple policies; policies can be assigned to multiple users.

Join Table: `user_policies`

Attribute	Data Type	Relation	Description
user_id	UUID	FK to users.user_id (owning side)	User assigned this policy
policy_id	UUID	FK to policies.policy_id	Policy assigned to user

- **Owning Side:** User is the owner — e.g., user can receive multiple policies.

b) policies ↔ invites (M:N)

Purpose: Invites can carry multiple policy assignments.

Join Table: `invite_policies`

Attribute	Data Type	Relation	Description
invite_id	UUID	FK to invites.invite_id (owning side)	Invite being configured
policy_id	UUID	FK to policies.policy_id	Policy linked for the invite

- **Owning Side:** Invite is the owner — An invite is configured with one or more policies.

Entity Relationship Overview

- **users:** The main table for all people in the system, including learners, employees, and organization owners.
- **learners & employees:** Specialized user types. Learners usually have fixed permissions, while employees can have multiple roles and custom permissions.
- **org_owner:** A special type of employee with highest organizational responsibilities.
- **policies:** Sets of permissions stored as JSON. These define what a user can do and are assigned to users or invites as permission groups.
- **policy_catalogue:** The master list of all possible permissions and actions available in the system. Policies are built from this catalogue to ensure consistency and control.
- **invites:** Manage onboarding by sending invitations with assigned roles and policies, guiding new users through registration.
- **history:** Records all important changes to users, policies, and other data for auditing and compliance.

Workflow Example: Inviting a New Employee User

1. Admin selects/creates a **policy** (group) according to the access needed for the new user, referencing available modules/actions in **policy_catalogue**.

2. Admin creates an **invite**, assigns **roles**, and attaches the corresponding **policy** or set of policies (via join table if needed).
3. The system generates a token and sends the invite to the user.
4. Upon registering (accepting the invite), the user is inserted into **users** (and **employees**), and the assigned policies are linked via **user_policies**.
5. All changes (invite, user registration, policy assignment) are tracked in the **history** table for audit.

Key Notes and Conventions

- **PK** = Primary Key, **FK** = Foreign Key.
- Enums should be well-defined for roles, statuses, actions, etc.
- Owning side: The table where the relation originates & is managed.
- Soft deletes: Use `is_deleted` for logical removal without losing data.
- All tables include audit columns for who/when created/updated.